Tela Detalhe do Pedido

Etapa 1 — Conferir a navegação da tela Meus Pedidos

No botão Ver detalhes de cada card de meus-pedidos.tsx, confira que o id_venda é enviado na navegação. Se já implementou o guia anterior, mantenha como está:

```
onPress={() => router.push({
  pathname: "/detalhe-pedido",
  params: { id: String(pedido.id_venda) },
})}
```

Etapa 2 — Ajustar imports de detalhe-pedido.tsx

Substitua os dois imports iniciais e acrescente AsyncStorage e API/IMAGEM. Preserve todos os outros imports, inclusive Linking, estilos e FooterScreen.

```
import { useState, useCallback } from "react";
import { router, useLocalSearchParams, useFocusEffect } from "expo-router";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { API, IMAGEM } from "@/config/api";
```

Etapa 3 — Criar tipos para os dados da API

Antes de export default function DetalhePedidoScreen(), adicione os tipos abaixo. Os campos opcionais podem vir como null.

```
type ItemPedido = {
  id_item: number;
  qtde_item: number;
  valor_unit_item: number;
  produto: { nome_produto: string; foto_produto: string | null } | null;
};
type Pedido = {
  id_venda: number;
  data_venda: string;
  status_venda: string;
  status_descricao_venda: string;
  itens: ItemPedido[];
  subtotal_venda: number;
  valor_desconto_venda: number;
  valor_venda: number;
  forma_pagamento_venda: "PIX" | "CARTAO" | "DINHEIRO";
  entrega_venda: "SIM" | "NAO";
  data_entrega_venda: string | null;
  observacao_venda: string | null;
  endereco: { nome_endereco: string; endereco: string; numero: string } | null;
  cupom: { codigo_cupom: string } | null;
};
```

Etapa 4 — Trocar os produtos fictícios por estados

Dentro de DetalhePedidoScreen(), exclua o array const itensResumo = [...] e insira:

```
const { id } = useLocalSearchParams<{ id: string }>();
const [pedido, setPedido] = useState<Pedido | null>(null);
const [carregando, setCarregando] = useState(true);
const [erro, setErro] = useState("");
```

Etapa 5 — Consultar o pedido na API

Logo depois dos estados, antes do return, crie a função. O token vem do AsyncStorage e o id é recebido da listagem. useFocusEffect atualiza ao abrir/retornar à tela.

```
const carregarPedido = useCallback(async () => {
  setCarregando(true);
  setErro("");
  try {
    if (!id || !/^\d+$/.test(id)) throw new Error("Pedido inválido.");
    const token = await AsyncStorage.getItem("token");
    if (!token) throw new Error("Sessão expirada. Faça login novamente.");
    const resposta = await fetch(`${API}/pedidos/${id}`, {
      headers: { Accept: "application/json", Authorization: `Bearer ${token}` },
    });
    const json = await resposta.json();
    if (!resposta.ok || !json.success || !json.data) {
      throw new Error(json.message || "Não foi possível carregar o pedido.");
    }
    setPedido(json.data);
  } catch (e) {
    setPedido(null);
    setErro(e instanceof Error ? e.message : "Erro ao carregar pedido.");
  } finally {
    setCarregando(false);
  }
}, [id]);

useFocusEffect(useCallback(() => { carregarPedido(); }, [carregarPedido]));
```

Etapa 6 — Criar formatadores e acompanhar o status

Ainda antes do return, adicione os formatadores. A trilha atual do layout tem três etapas e representa o fluxo de entrega. Para retirada, use o texto “Pronto para retirada” no lugar de “Saiu para entrega”.

```
const moeda = (valor: number) => Number(valor).toLocaleString("pt-BR", {
  style: "currency", currency: "BRL",
});
const dataBR = (valor?: string | null) => {
  if (!valor) return "Não informada";
  const data = new Date(valor);
  return Number.isNaN(data.getTime()) ? "Não informada" :
    data.toLocaleString("pt-BR", { dateStyle: "short", timeStyle: "short" });
};
const etapas = ["EM_PREPARO", "EM_ENTREGA", "ENTREGUE"];
const indiceEtapa = pedido?.status_venda === "PRONTO" ? 0 :
  pedido?.status_venda === "CONFIRMADO" ? -1 :
  pedido?.status_venda === "PENDENTE" ? -1 :
  etapas.indexOf(pedido?.status_venda ?? "");
const statusFinal = pedido?.status_venda === "ENTREGUE";
const statusCancelado = pedido?.status_venda === "CANCELADO";

const estiloEtapa = (etapa: number) => {
    if (statusCancelado) return {};

    if (indiceEtapa > etapa) {
      return detalhePedidoStyle.etapaFinalizada;
    }

    if (indiceEtapa === etapa) {
      return detalhePedidoStyle.etapaEmAndamento;
    }

    return {};
  };
```

.

Etapa 7 — Exibir carregamento e erro antes do conteúdo

Dentro de <View style={detalhePedidoStyle.main}>, envolva todos os blocos atuais (de areaPedido até botoesDetalhes) nesta condição. Assim, o JSX não acessa pedido enquanto ele é null.

```
{carregando ? (
  <Text>Carregando pedido...</Text>
) : erro ? (
  <Text>{erro}</Text>
) : pedido ? (
  <>
    {/* MANTENHA AQUI os blocos visuais atuais, adaptados nas etapas 8 a 11 */}
  </>
) : (
  <Text>Pedido não encontrado.</Text>
)}
```

Etapa 8 — Trocar número, data e status no cabeçalho

No bloco areaPedido, substitua apenas os textos fixos Pedido #1035, Hoje, 08:00 e Saiu para entrega:

```
<Text style={detalhePedidoStyle.numeroPedido}>
  Pedido #{pedido.id_venda}
</Text>
<Text style={detalhePedidoStyle.horarioPedido}>
  {dataBR(pedido.data_venda)}
</Text>
<Text style={detalhePedidoStyle.txtStatus}>
  {pedido.status_descricao_venda}
</Text>
```

O ícone delivery-laranja.png atual pode permanecer provisoriamente; ele não representa corretamente todos os status. Para um mapeamento de ícones, crie uma função por status em uma etapa visual posterior.

Etapa 9 — Atualizar acompanhamento e data desejada

Nos três ícones de acompanharPedido, aplique os estilos existentes com base no índice. Exemplo para a primeira etapa:

```
style={[
  detalhePedidoStyle.areaIconeEtapa,
  estiloEtapa(0),
]}
```

```
style={[
      detalhePedidoStyle.iconeEtapa,
      {
        tintColor:
          indiceEtapa > 0
            ? cores.verde
            : indiceEtapa === 0
              ? cores.laranja
              : cores.cinza,
      },
 ]}
```

Para a segunda e terceira etapas, troque as comparações por estiloEtapa(1) e estiloEtapa(2), respectivamente; para ENTREGUE, trate a última como concluída (statusFinal). Nas linhas, use linhaFinalizada quando o trecho já foi percorrido. Não deixe os ícones de concluído fixos quando o pedido estiver pendente.

Substitua os rótulos da segunda e terceira etapas conforme a modalidade e troque a previsão inventada pela data real desejada:

```
<Text style={detalhePedidoStyle.textoPedido}>
  {pedido.entrega_venda === "SIM" ? "Saiu para entrega" : "Pronto para retirada"}
</Text>
<Text style={detalhePedidoStyle.textoPedido}>
  {pedido.entrega_venda === "SIM" ? "Entrega realizada" : "Retirada concluída"}
</Text>
<Text style={detalhePedidoStyle.txtStatusAcompanhar}>Data desejada:</Text>
<Text style={detalhePedidoStyle.txtStatusTempo}>
  {dataBR(pedido.data_entrega_venda)}
</Text>
```

Etapa 10 — Mostrar os itens reais e valores

Na seção Resumo do pedido, substitua itensResumo.map pelo bloco abaixo, preservando os estilos de item, imagem, nome e valor:

```
{pedido.itens.map((item) => (
  <View style={detalhePedidoStyle.itemResumo} key={item.id_item}>
    <Image style={detalhePedidoStyle.imgProduto} resizeMode="stretch"
      source={{ uri: item.produto?.foto_produto
        ? `${IMAGEM}/${item.produto.foto_produto}`
        : `${IMAGEM}/produto/sem-imagem.png` }} />
    <Text style={detalhePedidoStyle.nomeProduto}>
      {item.qtde_item}x {item.produto?.nome_produto ?? "Produto"}
    </Text>
    <Text style={detalhePedidoStyle.valorProduto}>
      {moeda(Number(item.valor_unit_item) * Number(item.qtde_item))}
    </Text>
  </View>
))}
```

Nos campos de totais, troque os valores fictícios:

```
{/* Subtotal */}
{moeda(pedido.subtotal_venda)}
{/* Desconto */}
{moeda(pedido.valor_desconto_venda)}
{/* Total */}
{moeda(pedido.valor_venda)}
{/* Cupom, se houver */}
{pedido.cupom?.codigo_cupom ?? "Sem cupom"}
```

Importante: não mantenha “Entrega R$ 8,00” como valor real. A resposta documentada não possui campo específico para taxa de entrega. Remova/oculte areaValorEntrega até existir esse dado na API. O total não deve ser recalculado no aplicativo.

Etapa 11 — Forma de pagamento e observação

No texto Pix, substitua pelo valor retornado; no campo Observação, mostre o conteúdo ou a mensagem padrão. O ícone fixo pix.png deve ser mostrado somente quando a forma for PIX, ou removido nas demais formas.

```
<Text style={detalhePedidoStyle.textoForma}>
  {pedido.forma_pagamento_venda === "PIX" ? "Pix" :
   pedido.forma_pagamento_venda === "CARTAO" ? "Cartão" : "Dinheiro"}
</Text>

<Text style={detalhePedidoStyle.txtObservacao}>
  {pedido.observacao_venda?.trim() || "Nenhuma observação para seu pedido"}
</Text>
```